

# Suffix Array

Proyecto Final 1 — Anima tu Estructura de Datos

Aguirre Milla, Fernando  
Velásquez Díaz, Eliseo David  
Díaz Jara, Rolando David

CS2023 — Algoritmos y Estructuras de Datos, UTEC 2026-2  
Grupo 5

## 1 Estructura de datos y TDA

Un **Suffix Array** es un arreglo con las posiciones de inicio de todos los sufijos de un string  $s$ , ordenados lexicográficamente. Representa el Tipo de Dato Abstracto de un índice de texto: dado un patrón  $p$ , permite encontrar en qué posiciones de  $s$  aparece  $p$  sin recorrer  $s$  carácter por carácter. Se usa en motores de búsqueda de texto, bioinformática (alineamiento de genomas), detección de plagio/substrings repetidos y compresión de datos (Burrows–Wheeler Transform).

## 2 Implementación

La estructura se construye con *prefix doubling*: en la ronda  $k$  ya se conoce el orden relativo de los primeros  $k$  caracteres de cada sufijo (guardado en `rank`); comparar los primeros  $2k$  caracteres de dos sufijos se reduce a comparar el par  $(\text{rank}[i], \text{rank}[i + k])$ , y  $k$  se duplica en cada ronda hasta que todos los sufijos quedan distinguidos.

Adicionalmente se calcula el arreglo **LCP** (*Longest Common Prefix*) con el algoritmo de **Kasai** en  $O(n)$ , y se implementa **búsqueda binaria** (lower/upper bound) sobre el arreglo ya ordenado para encontrar el rango de sufijos que empiezan con un patrón dado.

No se usa ninguna librería de suffix arrays: todo el código (`src/suffix_array.hpp`) es una implementación propia del grupo, verificada con `src/test_suffix_array.cpp` contra una solución de fuerza bruta sobre miles de strings aleatorios y casos borde (string vacío, un carácter, string con todos los caracteres repetidos).

## 3 Herramientas usadas y reproducción del video

- **C++17** para la estructura de datos y las pruebas.
- **Manim Community** (Python) para la animación.
- Puente entre ambos: `src/trace_main.cpp` ejecuta el Suffix Array real sobre un string dado y emite un JSON con el arreglo, el LCP, las rondas de construcción y los pasos de la búsqueda binaria. La escena de Manim (`animation/scene.py`) llama a este binario en tiempo de render y dibuja exactamente lo que devuelve — ningún valor está puesto a mano en la animación.

Pasos para reproducir el video:

```

make                # compila build/sa_trace y build/test_suffix_array
make test           # corre las pruebas
cd animation
manim -pqh scene.py SuffixArrayScene

```

## 4 Contribuciones

- **Díaz Jara, Rolando David:** implementación de la estructura en C++ (`suffix_array.hpp`), pruebas contra fuerza bruta, y el programa de traza (`trace_main.cpp`) que expone rondas y pasos de búsqueda para la animación.
- **Aguirre Milla, Fernando:** animación en Manim (`scene.py`), ajuste de tiempos y renderizado del video final.
- **Velásquez Díaz, Eliseo David:** informe, revisión del video contra los requisitos del enunciado y entrega en Gradescope.

## 5 Análisis de complejidad

| Operación                                                        | Costo           |
|------------------------------------------------------------------|-----------------|
| Construcción (prefix doubling, <code>std::sort</code> por ronda) | $O(n \log^2 n)$ |
| LCP (Kasai)                                                      | $O(n)$          |
| Búsqueda de un patrón de largo $m$                               | $O(m \log n)$   |
| Espacio                                                          | $O(n)$          |

## 6 Evidencia

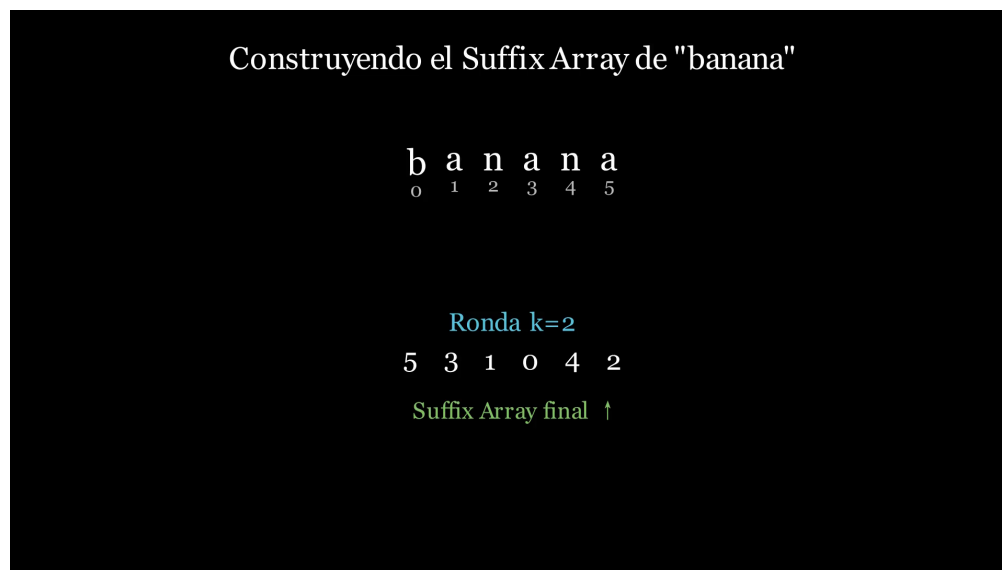


Figura 1: Fotograma del video mostrando la construcción del Suffix Array de "banana" por rondas.

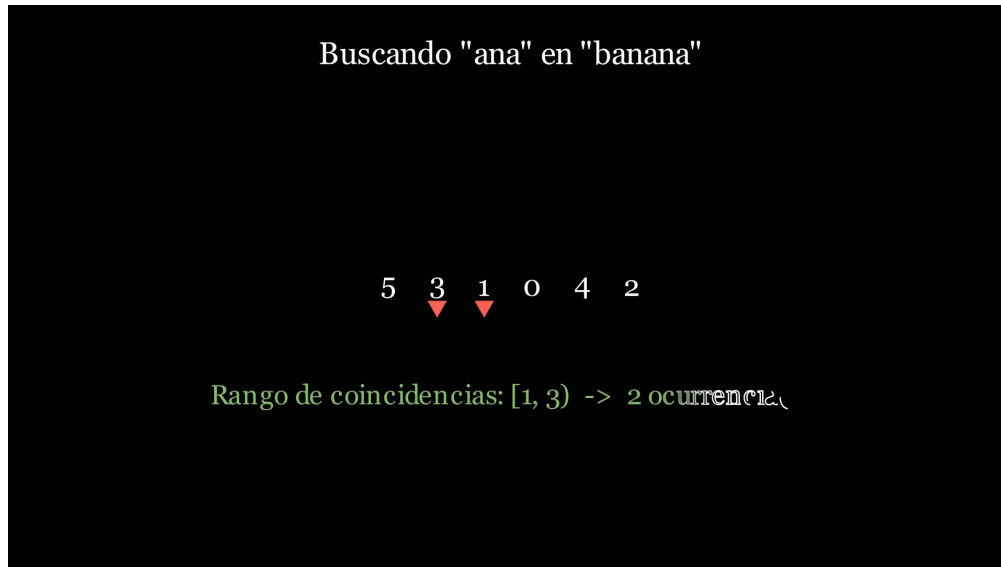


Figura 2: Fotograma del video mostrando la búsqueda binaria del patrón "ana".

Repositorio: <https://github.com/elmano777/aed-project>

Video: [media/suffix\\_array.mp4 \(en el repositorio\)](#)

## 7 Conclusiones

1. El Suffix Array permite convertir la búsqueda de patrones de texto, que ingenuamente costaría  $O(nm)$ , en una búsqueda binaria de  $O(m \log n)$ , a costa de una construcción inicial de  $O(n \log^2 n)$  que solo se paga una vez.
2. Separar la lógica real (C++) de la visualización (Manim) mediante una traza en JSON permitió verificar la implementación de forma independiente (con pruebas de fuerza bruta) y, al mismo tiempo, garantizar que la animación represente fielmente el comportamiento del algoritmo, sin necesidad de animar valores a mano.